TP2 · DEVOPS · UTN FRRE 2026 · GRUPO 4

DeployFlow

Guía del equipo: qué vamos a construir, quién hace qué y cómo trabajar los cuatro a la vez sin esperarnos.

> **En una frase:** vamos a construir una mini plataforma que ejecuta «pipelines» (como los de CI/CD) **en segundo plano**, y que nos deja **ver en vivo por dónde pasa cada pedido, cuánto tarda y dónde se rompe**.

Qué nos piden

Tema 10 · Mensajería

¿Cómo desacoplar una tarea lenta del camino principal de una solicitud?

Hay que mostrar: una API publica un trabajo, otro programa lo procesa con demora y la API responde sin esperar a que termine.

Tema 4 · Observabilidad

¿Cómo seguir una solicitud que atraviesa varios servicios?

Hay que mostrar: un identificador que viaja por las dos APIs (y por la cola) y una pantalla con el recorrido completo, con tiempos y errores.

Con **un solo flujo** mostramos los dos temas: la cola evita que la API se quede esperando, y la observabilidad nos deja ver qué pasó en el medio.

Lo esencial en 30 segundos

- **Somos 4 y trabajamos a la vez.** Cada uno tiene su parte, sus carpetas y sus archivos. Nadie edita los de otro.
- **Nadie espera a nadie.** Primero nos ponemos de acuerdo, por escrito, en cómo se hablan las piezas (el «contrato», Anexo A). Después cada uno usa versiones de mentira de las piezas vecinas hasta que las reales estén listas.
- **Todo va en el mismo repo** (`DevOps_Practica/lab2`). Se integra de a poco, con pedidos de cambio (PR) chicos y seguidos, no con un gran empalme al final.
- **Todo se levanta con un solo comando:** `docker compose up --build -d`.

Qué hay en esta guía

2 · La idea con una pizzería	6 · Cómo trabajamos sin esperarnos
3 · El recorrido de un pedido	7 · Orden recomendado y prioridades
4 · Tecnologías y demo en 5 pasos	8 · Reglas del repo
5 · Las 4 partes (backend, frontend, dependencias)	9 · Anexo A (contrato) · 10 · Glosario y pendientes

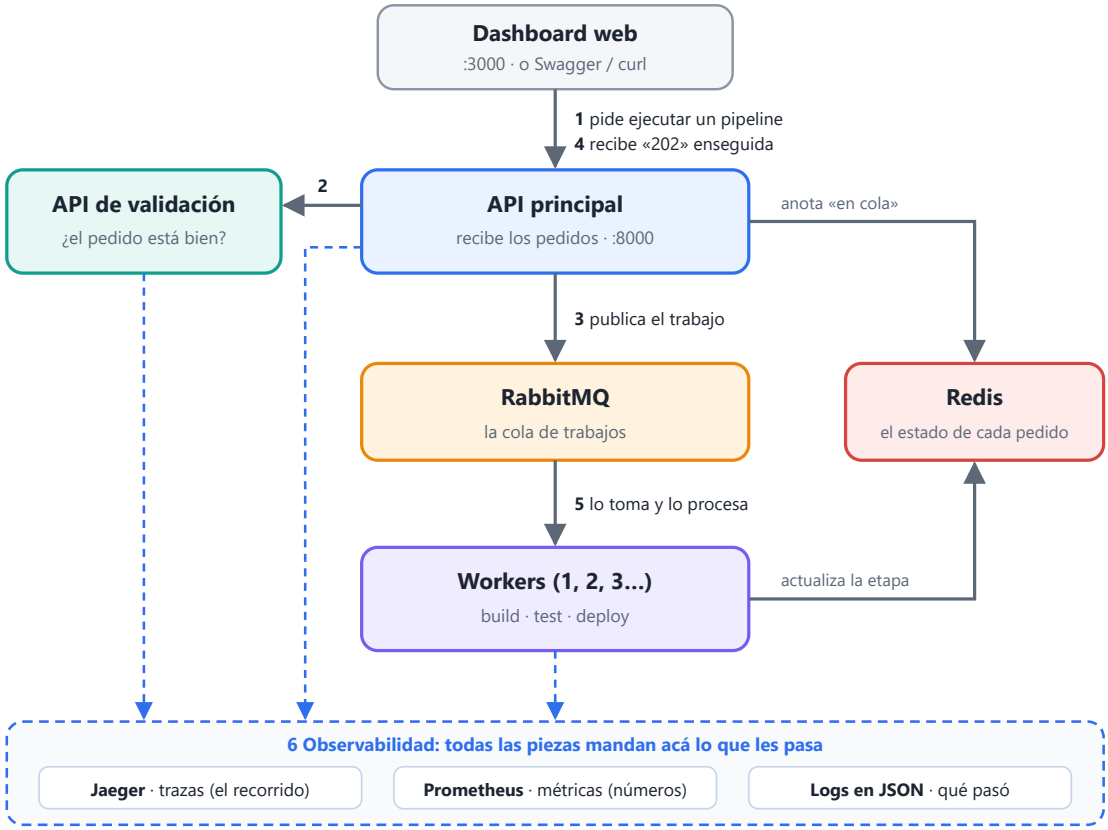
Versión 1 · 22/10/2026 · Los casilleros «Responsable» se completan cuando el grupo reparta las partes.

La idea, con una pizzería

Imaginá una pizzería. El mozo anota tu pedido, pregunta al encargado si se puede hacer, te da un número y **vuelve a atender mesas** sin quedarse esperando el horno. La cocina prepara los pedidos de a uno. Una pizarra dice en qué etapa va cada uno. Eso es exactamente DeployFlow:

En la pizzería	En DeployFlow
Vos, que hacés el pedido	Vos, desde el dashboard web o desde Swagger
El mozo, que anota y sigue atendiendo	**API principal**: responde «202, pedido aceptado» al instante
El encargado, que chequea el pedido	**API de validación**: ¿el pedido tiene sentido?
Los pedidos colgados en la cinta	**Cola** en RabbitMQ
Los cocineros	**Workers** (uno o varios): simulan build, test y deploy
La pizarra con «en el horno»	**Redis**: el estado de cada pedido
El número de pedido que lo acompaña a todos lados	El **trace_id**: un identificador que viaja por todas las piezas
El cuaderno, el contador y el recorrido del pedido	**Logs**, **métricas** y **trazas**

El mapa de piezas



Línea llena: el camino del pedido. Punteada azul: datos de observabilidad de la API, la validación y los workers.

El recorrido de un pedido, paso a paso

1. **Pedís ejecutar un pipeline.** Desde el dashboard o Swagger indicás el proyecto, las etapas (build, test, deploy), cuántos segundos dura cada una y, si querés, que **falle en test**.

2. **La API principal consulta a la API de validación.** Ella revisa que el nombre del proyecto y las etapas tengan sentido. Si algo está mal, responde `422` con los errores.

3. **Si está bien,** la API anota el pedido como «en cola» en Redis y **publica un mensaje en RabbitMQ**. Al mensaje le pega el `trace_id`, para no perder el hilo.

4. **Responde enseguida «202 Accepted»** con un `job_id` y el `trace_id`. No espera a que termine nada: queda libre para atender más pedidos.

5. **Un worker toma el mensaje** y simula build → test → deploy con demoras de unos segundos. Va anotando en Redis en qué etapa está. Si pediste que falle en test, el pipeline termina en `failed`.

6. **Mientras tanto, la observabilidad lo registra todo:** logs con el `trace_id`, números en Prometheus y la traza completa en Jaeger. El dashboard lee el estado y muestra el avance en vivo.

> **Ojo con el «202».** Significa «recibí tu pedido», **no** «terminó bien». La API puede responder 202 y, varios segundos después, el worker registrar un error. Por eso hace falta observabilidad: para enterarnos de lo que pasó *después* de que la API ya contestó.

Las tres «patas» de la observabilidad, en simple

Qué es	Qué pregunta responde	Ejemplo en nuestro proyecto
Logs	¿Qué pasó?	«El test del pipeline 42 falló» (una línea de texto, con el `trace_id`)
Métricas	¿Cuánto y con qué frecuencia pasa?	«8 pipelines fallidos sobre 100»; cuántos mensajes hay en la cola; cuánto tarda un job
Trazas	¿Dónde y por qué camino pasó?	Una línea de tiempo en Jaeger: API → validación → cola → worker → etapa «test» en rojo

La clave es el `trace_id`: es el mismo número en la respuesta de la API, en los logs del worker y en Jaeger. Con ese número seguimos un pedido de punta a punta.

Tecnologías, en palabras simples

Tecnología	Para qué la usamos	Dónde se ve
Python + FastAPI	Escribir las dos APIs (la principal y la de validación). Genera Swagger solo.	:8000 y :8001
RabbitMQ	La cola: guarda los trabajos hasta que un worker los tome.	:15672
Python + Pika	El worker. Pika es la librería para hablar con RabbitMQ.	logs del contenedor
Redis	La pizarra rápida con el estado de cada pedido.	`redis-cli`
React + Vite	El dashboard: formulario, lista en vivo y estadísticas.	:3000
OpenTelemetry	Las «cámaras»: hace que cada servicio genere trazas.	—
Jaeger	La pantalla para ver el recorrido de un pedido.	:16686
Prometheus	Guarda números a lo largo del tiempo (cuántos, cuánto tardan).	:9090
Docker Compose	Levanta todas las piezas juntas con un solo comando.	terminal

La demo en 5 pasos (lo que vamos a mostrar)

1. **Ejecutar un pipeline.** Un `POST /pipelines`, la validación lo aprueba y la API responde `202` con un `job_id`.

2. **Mostrar que es asíncrono.** El worker tarda varios segundos a propósito. Mientras tanto la API ya respondió y acepta más pedidos; el estado se consulta con `GET /pipelines/{id}`.

3. **Ver la traza distribuida.** En Jaeger, el mismo `trace_id` atraviesa la API, la validación y el procesamiento del mensaje.

4. **Provocar un fallo controlado.** Un pipeline configurado para fallar en test: se ve el error en los logs, el span en rojo en Jaeger y el estado `failed`.

5. **Escalar.** Mandamos muchos trabajos, vemos crecer la cola y pasamos de 1 a 3 workers. Comparamos la velocidad y las métricas antes y después.

> **Qué queda fuera:** login de usuarios, despliegues reales y un frontend complicado. Si el dashboard no llega a tiempo, Swagger alcanza para la demo.

Las 4 partes de un vistazo

Parte	Backend	Frontend	Depende de
P1 **API**	Validador y API principal: crear, ver y listar pipelines, estadísticas	— (deja Swagger listo para el dashboard)	P2 (cola y estado reales) y P3 (trazas), recién al integrar
P2 **Mensajería**	Cola, worker, etapas simuladas, estado en Redis, reintentos	—	Nada para empezar. P3 le da las trazas al integrar
P3 **Observabilidad**	Trazas, logs JSON, métricas, Jaeger, Prometheus	Define el link a la traza y las consultas útiles	Nada. Entrega funciones que P1 y P2 enchufan
P4 **Front + plataforma**	Compose de todo, nginx, scripts de demo, README	**Dashboard React**	La API real de P1 y los Dockerfiles de P1/P2, recién al integrar

«Recién al integrar» significa que nadie se frena: hasta ese momento cada uno usa versiones de mentira de sus vecinos (ver la sección 6).

P1 API (principal + validación)

Ser la puerta de entrada: recibir el pedido, chequearlo y dejarlo en la cola sin hacer esperar a nadie. Tamaño: mediano · Responsable: _____

Backend
- **Validador:** servicio chico que revisa el nombre del proyecto, las etapas y los segundos.
- **API principal:** crear pipeline, ver uno, listar, estadísticas; responde 202 / 422 / 503.
- **Swagger** con ejemplos listos para probar (uno OK y uno que falla en test).

Frontend No tiene. Deja la API documentada para que P4 haga la pantalla.

Depende de P2 (publicar en la cola y guardar estado «de verdad») y P3 (trazas), solo al integrar. Hasta entonces usa un guardado en memoria y un «publicador de mentira».

Alimenta a P4 (el dashboard usa tu API). Al final hace los tests de punta a punta y el CI.

Empezá hoy 1) Las reglas del validador como funciones puras con tests. 2) `POST` / `GET` de la API con el guardado falso. 3) Los ejemplos de Swagger.

Listo cuando Responde 202 en menos de 0,3 s y el mensaje aparece en la cola.

P2 Mensajería, worker y estado

Ser la cocina: tomar los pedidos de la cola, hacerlos con demora y anotar en qué etapa van. Tamaño: grande · Responsable: _____

Backend
- **Cola:** la principal y la de mensajes fallidos; publicador con confirmación.
- **Worker:** simula build → test → deploy, con ack manual y de a un trabajo por vez.
- **Estado en Redis:** etapa actual, tiempos, errores; latido de cada worker.
- Extra: reintentos con espera y cola de mensajes muertos (DLQ).

Frontend No tiene. Lo que guardás en Redis es lo que el dashboard muestra.

Depende de Nada para empezar. P3 le entrega las trazas y métricas al integrar.

Alimenta a P1 (publicar y leer el estado) y P4 (cola y workers vivos para las estadísticas).

Empezá hoy 1) El ejecutor de etapas con tests (sin RabbitMQ). 2) RabbitMQ y Redis con compose y un script que mande mensajes de prueba. 3) El consumidor con ack manual.

Listo cuando Un pedido de prueba termina en Redis; el que falla en test queda `failed`; si matás un worker a mitad de un trabajo otro lo termina; con 3 workers se reparten la cola.

Las 4 partes (continuación)

P3 Observabilidad

Ser las cámaras y los contadores: que todo lo que pase quede registrado y se pueda seguir con un solo número.
Tamaño: grande · Responsable: _____

Backend
- **Librería común:** trazas con OpenTelemetry, logs en JSON con `trace_id` y métricas.
- **Propagación por la cola:** el `trace_id` viaja dentro del mensaje (a mano, para poder explicarlo).
- **Prometheus** (que encuentre 1 o N workers), **Jaeger** y una hoja de consultas para la demo.

Frontend Define el enlace a la traza en Jaeger y las consultas útiles; P4 los muestra.

Depende de Nada. Entrega funciones con los mismos nombres que los «stubs» del esqueleto, y P1/P2 las usan sin cambiar su código.

Alimenta a P1 y P2 (las funciones), P4 (las configuraciones del compose).

Empezá hoy 1) El formateador de logs JSON con test. 2) Un test de que el `trace_id` sobrevive a «meter y sacar» del mensaje. 3) Levantar Jaeger y Prometheus y validar la configuración.

Listo cuando Una sola traza en Jaeger muestra API → validación → cola → worker → etapas, con el mismo número que sale en la respuesta y en los logs.

P4 Frontend + plataforma

Ser la vidriera y el pegamento: la pantalla para ver todo en vivo y el compose que lo levanta con un comando.
Tamaño: grande · Responsable: _____

Frontend
- **Dashboard React** (de una sola pantalla): formulario con atajos *Ejecutar OK*, *Forzar fallo en test* y *Lanzar 10*.
- **Lista en vivo** (se actualiza cada 2 s) con estado, etapas y duración, y el enlace *Ver traza en Jaeger*.
- **Panel de estadísticas:** pedidos por estado, cola, workers vivos; enlaces a RabbitMQ, Jaeger y Prometheus.

Backend
- **docker-compose** con los 8 servicios, chequeos de salud y orden de arranque.
- **nginx** que reenvía `/api` a la API.
- **Scripts de demo** (ok, fail, burst, scale) y README con el paso a paso.

Depende de La API real de P1 y los Dockerfiles de P1/P2, solo al integrar. Hasta entonces trabaja con un servidor de mentira que devuelve los ejemplos del contrato.

Alimenta a A todos: es quien integra el conjunto y arma el README y la guía del coloquio.

Empezá hoy 1) Proyecto Vite + React y el servidor de mocks. 2) Formulario y lista contra los mocks. 3) Compose solo con la infraestructura.

Listo cuando Un clon limpio + `docker compose up --build -d` deja todo andando y el dashboard funciona sin pasos manuales.

Repartir el esfuerzo: P1 es la más liviana. Cuando termina su núcleo, toma los **tests de punta a punta** y el **CI**, que son de P4, para que nadie se ahogue al final.

Cómo trabajamos sin esperarnos

El truco son **tres ideas** que se combinan:

1 · Contrato primero

Escribimos **de antemano** cómo se hablan las piezas: qué URLs hay, qué JSON viaja, cómo se llama la cola, qué nombres tienen las métricas. Está en el Anexo A y se congela en el Paso 0.

2 · Dobles de prueba

Cada uno trabaja contra una **versión de mentira** de sus vecinos (guardado en memoria, publicador falso, servidor de mocks) que cumple el mismo contrato.

3 · Interruptores

Con variables de entorno pasamos de lo falso a lo real **sin tocar código**: `STATE_BACKEND`, `PUBLISHER` y `TELEMETRY`.

Qué usa cada uno mientras los demás no terminaron

Parte	Usa de mentira / ya disponible	Trabaja sola con
P1 API	Guardado en memoria, publicador falso y telemetría apagada (`memory` / `fake` / `off`)	El servidor local, `pytest` y Swagger
P2 Worker	RabbitMQ y Redis reales del compose de infraestructura; mensajes que manda un script, sin la API	`pytest` y el worker en una terminal
P3 Observabilidad	Una app de juguete y un «registro en memoria» de trazas; Jaeger y Prometheus reales	`pytest` y la validación de la config
P4 Front + plataforma	Un servidor de mocks que responde con los ejemplos del contrato y simula el avance de las etapas	`npm run dev:mock` y `docker compose config`

La escalera de integración: cada escalón se «enciende» cambiando una variable

N0 · Contrato

Esqueleto del repo y contrato congelado. Lo hace **una** persona.

N1 · Solos

Cada parte funciona sola con sus dobles. Los 4, en paralelo.

N2 · Núcleo

API ↔ cola ↔ worker reales (`redis` + `rabbit`). Entra primero P2, después P1.

N3 · Observa

P3 reemplaza los stubs; `TELEMETRY=on` . P1 y P2 no cambian código.

N4 · Todo

El dashboard apunta a `/api` real y el compose levanta los 8 servicios. Lidera P4.

Cómo se acopla todo en un mismo repo: cada uno sube **PR chicos a `main` en cuanto algo funciona** (como mínimo una vez por día de trabajo) y `main` siempre tiene los tests en verde. Antes de ponerte a trabajar, hacé `git pull --rebase origin main` . Así los resultados de los cuatro se van sumando de a poco y nadie espera un «gran empalme».

Orden recomendado

0 **Paso 0 · Contrato y esqueleto (N0).** Es lo **único secuencial** y dura poco. Una persona crea `lab2/` con la estructura, las librerías de prueba y los ejemplos del contrato. Mientras tanto, los otros tres leen esta guía, instalan lo que necesitan y revisan el Anexo A.

1 **Paso 1 · Arrancan los 4 a la vez (N1).** No hay orden entre ellos: cada uno trabaja en sus carpetas con sus dobles.

2 **Paso 2 · Núcleo asíncrono (N2).** Se integra primero **P2** (cola, worker y estado reales) y después **P1** (API real). Es el primer «hola mundo» de punta a punta.

3 **Paso 3 · Observabilidad (N3). P3** enciende trazas, métricas y logs. P1 y P2 solo cambian una variable.

4 **Paso 4 · Todo junto (N4). P4** conecta el dashboard real y cierra el compose completo; **P1** toma los tests de punta a punta y el CI.

5 **Paso 5 · Ensayo.** Ensayar la demo dos veces, congelar el README y la guía del coloquio (el coloquio es **individual**: cada uno tiene que poder explicar todo el proyecto).

Si falta tiempo, esta es la prioridad

1.º · P2
Mensajería: es uno de los dos temas.

2.º · P3
Observabilidad: el otro tema.

3.º · P1
Sin API no hay flujo.

4.º · P4
El dashboard es lo primero que se recorta: Swagger lo reemplaza.

Quién necesita a quién, y cuándo

Parte	Necesita…	Cuándo	Mientras tanto
P1	El publicador y el estado reales de P2	N2	Guardado en memoria y publicador falso
P1, P2	Las funciones de trazas, logs y métricas de P3	N3	Funciones vacías («stubs») que no hacen nada
P4	La API real de P1	N4	Servidor de mocks con los ejemplos del contrato
P4	Los Dockerfiles de P1 y P2, y la config de Prometheus de P3	N4	Compose probado solo con la infraestructura
P2, P3	Nada de nadie para empezar	—	—

Fijate que **nadie depende de un resultado terminado para arrancar**: las dependencias aparecen recién en los escalones N2 a N4, y para entonces cada parte ya funciona sola.

Reglas del repo (para no pisarnos)

Cada parte tiene sus carpetas

Carpeta o archivo (dentro de `lab2/`)	De quién es
`docs/`	Paso 0: contrato, ejemplos de respuestas, esta guía
`common/contract.py`	Paso 0 (se congela)
`common/messaging.py`, `common/state.py`	P2
`common/observability.py`, `logging.py`, `metrics.py`	P3
`services/api/` y `services/validator/`	P1
`services/worker/`	P2
`services/web/`	P4
`observability/` (Prometheus, Jaeger, consultas)	P3
`docker-compose.yml`, `scripts/`, `README.md`	P4
`tests/e2e/` y `.github/workflows/lab2-ci.yml`	P1 (al final)

- **No se edita un archivo de otra parte.** Si necesitás algo, lo pedís o cambiás el contrato (ver abajo).
- **Cambiar el contrato** (un endpoint, el mensaje, un nombre de métrica) se avisa a todos y va en un PR chico que se mergea primero; los demás hacen `rebase`.
- **Dependencias nuevas:** las del Paso 0 ya están fijadas. Si necesitás otra, agregala al `requirements.txt` de **tu** servicio.
- **docker-compose.yml** lo toca solo P4. Si necesitás un puerto o una variable, la sumás al contrato y se lo avisás.

Git, en cinco reglas

1. Una rama por parte: `lab2/p1-apis`, `lab2/p2-worker`, `lab2/p3-observabilidad`, `lab2/p4-front-plataforma`. Todas salen del Paso 0.
2. PR a `main` chicos y seguidos. `main` siempre con los tests en verde.
3. `git pull --rebase origin main` antes de empezar a trabajar cada día.
4. Mensajes de commit tipo `feat(lab2): validador con las reglas de etapas`.
5. **No tocar** `lab1` y agregar solo rutas de `lab2/` (`git add lab2`). No subir contraseñas ni archivos `.env` (el `.env.example` sí).

Qué instalar antes de empezar

Herramienta	Quién la necesita
Docker Desktop (con Compose)	Los 4
Git	Los 4
Python 3.12 o superior	P1, P2, P3 (y P4 para los scripts de demo)
Node 20 o superior	P4

Anexo A · El contrato resumido

Con esto alcanza para empezar **aunque todavía no exista el esqueleto del repo**. El Paso 0 lo deja escrito en código y con ejemplos de cada respuesta.

Endpoints de la API principal (puerto 8000)

```
POST /pipelines           → 202 { job_id, status:"queued", trace_id }   (+ cabecera Location)
                            422 { detail:{ errors:[…] } }               (el validador lo rechazó)
                            503                                         (falla el validador o la cola)
GET  /pipelines/{id}      → { job_id, project, status, current_stage, stages:[…], attempts,
                              trace_id, submitted_at, finished_at, error }      · 404 si no existe
GET  /pipelines?limit=20  → { items:[ …lo mismo que arriba… ] }   (el más nuevo primero)
GET  /stats               → { counts:{queued,running,succeeded,failed}, queue_depth, workers }
GET  /health   GET /metrics
POST /validate  (en el validador)  → { valid, errors:[] }
```

Cuerpo del pedido y estados

```
{ "project": "shop-frontend", "branch": "main",
  "config": { "stages": ["build","test","deploy"], "stage_seconds": 3, "fail_at": null } }

status:  queued | running | succeeded | failed
stage:   { name, status, started_at, duration_s, error }
Reglas:  project = minúsculas, números y guiones (2 a 41 caracteres) · stages ⊂ build, test, deploy en ese orden
         stage_seconds entre 1 y 30 · fail_at = null o una de las etapas
```

Cola (RabbitMQ) y estado (Redis)

```
Exchange  pipelines  (routing key pipeline.run)  →  cola  pipeline.jobs   (duradera, ack manual, de a 1 por worker)
Cola de mensajes fallidos:  pipeline.jobs.dead     ·  reintentos (extra): pipeline.jobs.retry (espera 5 s)
Mensaje:  { job_id, project, branch, config, submitted_at }   cabeceras: traceparent, x-attempt

Redis:  job:{id} = estado en JSON (se borra a las 24 h) · jobs:recent = índice por fecha
        worker:{host} = latido de cada worker (se renueva cada 5 s y vence a los 15 s)
```

Nombres que tienen que coincidir en todas las partes

```
Métricas:  http_requests_total · http_request_duration_seconds · pipelines_submitted_total
           pipeline_jobs_total{status} · pipeline_job_duration_seconds · pipeline_stage_duration_seconds{stage}
           pipeline_stage_failures_total{stage} · worker_jobs_in_progress · worker_messages_redelivered_total
Servicios de trazas:  api · validator · worker       Enlace a la traza:  {JAEGER_URL}/trace/{trace_id}
Log (una línea JSON):  ts, level, service, msg, trace_id, span_id, job_id?, stage?, error?
Interruptores:  STATE_BACKEND=memory|redis · PUBLISHER=fake|rabbit · TELEMETRY=on|off
Puertos:  web 3000 · api 8000 · validator 8001 · RabbitMQ 5672 y 15672 · Redis 6379
          Jaeger 16686 · Prometheus 9090 · el worker no publica puertos (se escala con --scale)
```

Glosario

Palabra	Qué quiere decir, en simple
Pipeline	Una serie de etapas que se ejecutan en orden (build, test, deploy). Acá es simulado.
Cola	Una fila de trabajos pendientes. Quien manda no espera; quien procesa toma el siguiente cuando puede.
Worker	El programa que toma trabajos de la cola y los hace. Se pueden prender varios para ir más rápido.
Ack	El «recibido, ya lo terminé» que el worker le manda a la cola. Si el worker se cae antes, la cola le da el trabajo a otro.
DLQ	Cola de mensajes muertos: donde van a parar los trabajos que no se pudieron procesar, para revisarlos después.
Asíncrono	Pedir algo y seguir con lo tuyo, sin esperar la respuesta final.
202 Accepted	Respuesta HTTP que significa «recibí tu pedido, lo voy a procesar». No dice que terminó.
Traza / trace_id	El recorrido completo de un pedido por todas las piezas, identificado con un número único.
Span	Un tramo de una traza (por ejemplo «la etapa de test»), con su inicio, su duración y si falló.
Log	Una línea de texto que dice qué pasó. Los nuestros están en JSON y llevan el `trace_id`.
Métrica	Un número que se mide en el tiempo (cuántos pedidos, cuánto tardan, cuántos fallaron).
Contenedor	Una caja liviana con un programa y todo lo que necesita para correr igual en cualquier máquina.
Docker Compose	La receta que levanta todos los contenedores juntos y los conecta entre sí.
Contrato	El acuerdo escrito de cómo se hablan las piezas. Si todos lo cumplen, se pueden construir por separado.
Mock / doble de prueba	Una versión de mentira de una pieza, que responde lo que dice el contrato, para poder trabajar sin esperar la real.
PR	Pedido de cambio: se propone sumar tu rama a `main` para que el resto la revise y la vea.

Qué falta definir

1. **Quién toma cada parte** (los casilleros «Responsable» de las tarjetas).
2. **La consigna oficial del TP2, la rúbrica y la fecha de entrega.** Hay que pegarlas en el `AGENTS.md` del Paso 0 para que el trabajo apunte a lo que puntúa.
3. Si hay que publicar las imágenes en Docker Hub o desplegar en la nube, como en el TP1.
4. Si las diapositivas se entregan como archivo. El esquema de las 7 diapositivas está en el README.

> **Primer paso concreto:** una persona hace el Paso 0 (contrato y esqueleto). En cuanto esté en `main`, los cuatro arrancan en paralelo.